FIRST Tech Challenge 2026-2027

Programmer son robot FTC

Outils, capteurs, contrôle, mode autonome, vision et travail en équipe

Robotique FIRST France



Robotique *FIRST* France
Association loi 1901 à but non lucratif d'intérêt général
22, avenue Gambetta 69250 Neuville Sur Saône · N° RNA : W691104750 · N° SIREN 904 913 043

Table des matières

🚀 Avant de lire ce guide 4

💻 1. Choisir son outil de programmation 5

1.1. Blocks : accessible, mais pas limitant 5

1.2. OnBot Java 5

1.3. Android Studio avec Git et GitHub 6

1.4. Comparer et choisir 6

🧪 2. Commencer à apprendre sans robot : FTC Sim 8

🧩 3. Les bases : rappels et bonnes pratiques 10

3.1. Mettre à jour en début de saison 10

3.2. Ajouter des moteurs et des ports 10

3.3. Structurer ses OpModes 11

3.4. La télémétrie 11

3.5. Sauvegarder la configuration et les programmes 11

📡 4. Les capteurs 12

4.1. Les encodeurs 12

4.2. L'IMU 13

4.3. Les autres capteurs 13

🎛️ 5. Le contrôle en boucle fermée (PID) 14

5.1. Pourquoi mesurer pour corriger 14

5.2. Du tout ou rien au PID 14

5.3. Régler les constantes 15

🤖 6. Le mode autonome 16

6.1. Les timers 16

6.2. Se déplacer avec les encodeurs et l'IMU 16

6.3. Organiser le programme : la machine à états 16

6.4. Savoir où est le robot : l'odométrie 17

👁️ 7. La vision par ordinateur 18

7.1. Les AprilTags 18

7.2. Choisir une webcam 18
📁 8. Coder à plusieurs : Git et GitHub 19
8.1. Les notions à connaître 19
8.2. Mettre en place l'environnement Android Studio 19
8.3. Une organisation qui fonctionne 21
🧯 9. Dépannage 23
📚 Ressources supplémentaires 24
✅ Pour commencer dès maintenant 25

Avant de lire ce guide

Ce guide s'adresse aux équipes qui ont déjà un robot qui roule et qui veulent aller plus loin : choisir le bon outil de programmation, exploiter les capteurs, fiabiliser le mode autonome, utiliser la vision et coder à plusieurs sans se marcher dessus.

À savoir avant de commencer

• **Ce guide n'apprend pas la syntaxe Java.** De nombreuses ressources sont disponibles sur internet pour l'apprendre, par exemple le cours gratuit Apprenez à programmer en Java d'OpenClassrooms. Ici, on se concentre sur ce qui est propre à la FTC.

• **Le câblage du Control Hub** (batterie, moteurs, encodeurs, servomoteurs) est détaillé dans le guide « Robot de démarrage de A à Z ».

• **Ce que doit faire le mode autonome** se décide avec la stratégie de jeu : voir la partie Stratégie du guide « Gérer son projet ».

Le guide suit la progression d'une équipe :

1. **Choisir son outil** : Blocks, OnBot Java ou Android Studio (chapitre 1).
2. **Apprendre et consolider les bases** : FTC Sim, rappels et bonnes pratiques (chapitres 2 et 3).
3. **Mesurer et corriger** avec les capteurs et le contrôle en boucle fermée (chapitres 4 et 5).
4. **Automatiser** : mode autonome et vision par ordinateur (chapitres 6 et 7).
5. **Travailler en équipe** sur le code avec Git et GitHub (chapitre 8).

1. Choisir son outil de programmation

Trois outils permettent de programmer un robot FTC. Ils pilotent le même matériel et donnent accès aux mêmes capteurs : ce qui les distingue, c'est le confort d'écriture, le travail à plusieurs et la gestion des versions.

1.1. Blocks : accessible, mais pas limitant

Blocks est une programmation par blocs à assembler, proche de Scratch, directement dans le navigateur et sans installation. Son image de « langage pour débutants » est trompeuse : on peut aller très loin en Blocks.

- Moteurs, servomoteurs, encodeurs, déplacement à une position cible et contrôle en vitesse.
- Tous les capteurs : IMU, contact, couleur, distance.
- La vision par ordinateur : détection des AprilTags avec VisionPortal.
- Variables, fonctions, timers et machines à états pour structurer le mode autonome.
- Même la planification de trajectoire : l'odométrie et le suivi de trajectoire peuvent se programmer en Blocks.
- Le bouton **Show Java** affiche le code Java équivalent : une passerelle naturelle vers Java.

La vraie limite de Blocks : coder à plusieurs

• Chaque programme Blocks est un fichier unique stocké sur le Control Hub. Si deux élèves modifient le même programme en même temps, le dernier qui enregistre écrase le travail de l'autre.

• Il n'y a pas d'historique des versions : pour revenir à une version qui fonctionnait, il faut l'avoir téléchargée soi-même avant de la modifier.

• C'est la seule vraie limitation par rapport à un environnement de développement comme Android Studio : la complexité de coder à plusieurs sur le même code et de gérer les versions.

1.2. OnBot Java

OnBot Java permet d'écrire du Java directement dans le navigateur. Le code est stocké et compilé sur le Control Hub : aucune installation n'est nécessaire.

- **Avantages** : accès complet aux fonctions du SDK, code organisé en plusieurs fichiers et classes (par exemple une classe commune qui décrit le matériel du

robot, réutilisée par tous les programmes), fonctionne sur n'importe quel ordinateur avec un navigateur.

- **Limites** : éditeur simple (peu d'aide à la saisie, pas de vrai débogueur), code stocké sur le robot comme en Blocks, donc mêmes difficultés pour coder à plusieurs et garder un historique. Il faut connaître les bases de Java.

1.3. Android Studio avec Git et GitHub

Android Studio est l'environnement de développement professionnel utilisé pour créer des applications Android. L'équipe y ouvre le projet officiel FtcRobotController, récupéré sur GitHub, puis envoie le programme au Control Hub par câble USB ou en Wi-Fi.

- **Versionnage avec Git** : chaque modification est enregistrée dans un historique. On revient en quelques secondes à la version qui fonctionnait la veille.
- **Travail à plusieurs** : chaque programmeur travaille sur sa branche, puis les modifications sont fusionnées après test (chapitre 8).
- **Aide à l'écriture** : complétion automatique, erreurs signalées pendant la frappe, débogueur.
- **Bibliothèques externes** : suivi de trajectoire (Road Runner, Pedro Pathing), outils de réglage en direct et bibliothèques de la communauté.
- **Contraintes** : un ordinateur dédié et assez puissant, une installation plus lourde, un temps de prise en main, et une bonne connaissance de Java.

La mise en place pas à pas (GitHub, Android Studio, clonage du projet) est au chapitre 8.2. Pour aller plus loin : utiliser Android Studio (Game Manual 0, en français) et envoyer le code en Wi-Fi (REV).

1.4. Comparer et choisir

Critère	Blocks	OnBot Java	Android Studio
Installation	Aucune (navigateur)	Aucune (navigateur)	Logiciel sur un ordinateur dédié
Fonctions du robot	Moteurs, capteurs, vision, trajectoires	Tout le SDK	Tout le SDK + bibliothèques externes
Aide à l'écriture	Blocs guidés	Basique	Avancée (complétion, débogueur)
Coder à plusieurs	Difficile	Difficile	Facile (branches Git)
Historique des versions	Sauvegardes manuelles	Sauvegardes manuelles	Git
Prérequis	Aucun	Bases de Java	Bonne maîtrise de Java

Quel outil choisir ?

• **Rester en Blocks** tant qu'une ou deux personnes codent et que l'équipe n'a pas besoin de bibliothèque externe : ce n'est pas un handicap en compétition.

• **Passer à OnBot Java** pour écrire du texte et structurer le code en classes, sans rien installer.

• **Passer à Android Studio** quand plusieurs élèves codent en même temps ou que l'équipe veut une bibliothèque externe. Le moment conseillé est entre deux saisons. Ce n'est pas une obligation, mais prévoir d'abord un temps de prise en main (installer l'outil, cloner le projet, envoyer un premier programme, s'exercer avec Git) avant d'y basculer le code de compétition.

• **En attendant**, une équipe en Blocks ou en OnBot Java limite les risques en désignant un responsable par programme et en sauvegardant régulièrement ses programmes (chapitre 8).

Le lien avec le monde professionnel

• Git est l'outil de gestion de versions utilisé dans presque toutes les entreprises du logiciel : historique, branches, revue du code avant fusion. Un élève qui le maîtrise a déjà un pied dans le métier.

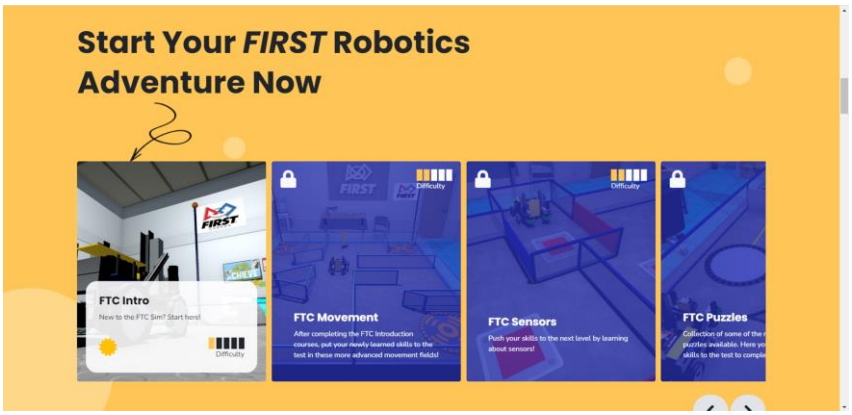
🧪 2. Commencer à apprendre sans robot : FTC Sim

Le robot est en construction, en réparation ou monopolisé par les pilotes ? Les nouveaux programmeurs peuvent commencer à apprendre avec FTC Sim, un outil gratuit en ligne : il suffit d'un navigateur, sans installation.

- Des cursus guidés pour apprendre la programmation FTC pas à pas, en Blocks ou en Java, accessibles en bas de la page d'accueil.
- Plusieurs élèves peuvent travailler en même temps, chacun sur son ordinateur : pratique pour former les nouveaux programmeurs de l'équipe.
- C'est un outil de formation : les réglages du programme de l'équipe (distances, temps, constantes du PID) se font toujours sur le vrai robot.



Page d'accueil de FTC Sim



Les cursus, accessibles en bas de la page d'accueil

Le lien avec le monde professionnel

• L'industrie simule avant de construire : les lignes de production et les robots industriels sont programmés et testés sur un double numérique (on parle de mise en service virtuelle) avant de toucher la vraie machine.

3. Les bases : rappels et bonnes pratiques

La mise en route du système de contrôle (REV Hardware Client, mises à jour, Wi-Fi, appairage), la configuration du hardware, la structure d'un OpMode, le premier programme, la manette et les servomoteurs sont détaillés pas à pas dans le guide « Robot de démarrage de A à Z ». Ce chapitre ne garde que les rappels et les habitudes qui comptent quand le code grossit.

3.1. Mettre à jour en début de saison

- Le manuel de compétition fixe chaque saison une version minimale du logiciel : mettre à jour le Driver Hub et le Control Hub avant la première compétition, et vérifier qu'ils ont la même version.
- Équipes sous Android Studio : mettre aussi à jour leur copie du projet FtcRobotController avec la dernière version du SDK (procédure REV).

La règle d'or de la configuration

• Le nom donné dans la configuration doit être **strictement identique** au nom utilisé dans le programme, majuscules comprises : « leftDrive » et « LeftDrive » sont deux noms différents.

• Détails : les erreurs courantes de configuration (REV).

3.2. Ajouter des moteurs et des ports

Le Control Hub offre 4 ports moteur et 6 ports servo. Quand le robot grandit, deux solutions existent :

Solution	Ce qu'elle apporte	À savoir
Expansion Hub	Un second Hub complet : 4 ports moteur avec encodeurs, 6 ports servo et des ports capteurs supplémentaires.	Se déclare dans la configuration comme un second Hub. Les moteurs gardent la régulation intégrée (RUN_USING_ENCODER, RUN_TO_POSITION).
REV SPARK Mini	Un contrôleur pour un moteur supplémentaire, branché sur un port servo (commande) et sur l'alimentation 12 V.	Petit et économique. Le moteur se commande en puissance, sans la régulation intégrée du Hub : la boucle de contrôle se programme soi-même. Pas de protection contre l'inversion de polarité.

Le manuel de compétition fixe le nombre maximal de moteurs et les contrôleurs autorisés : le vérifier chaque saison.

3.3. Structurer ses OpModes

- **TeleOp** pour la période pilotée, **Autonomous** pour la période autonome : donner des noms explicites pour que le pilote lance le bon programme sur le Driver Hub.
- **Avant le départ** (INIT) : configurer les moteurs, remettre les encodeurs à zéro, initialiser l'IMU, démarrer la caméra. **Pendant le match** : une boucle qui lit les capteurs, décide et commande, sans jamais rester bloquée.
- En Java, deux structures existent : **LinearOpMode**, qui se lit de haut en bas comme un programme Blocks, et **OpMode**, où le SDK appelle des méthodes init et loop en boucle. Voir LinearOpMode ou OpMode (Game Manual 0).

3.4. La télémétrie

L'outil de débogage numéro un

• **Telemetry.addData** prépare une ligne d'affichage (une étiquette et une valeur) ; **Telemetry.update** envoie les lignes sur l'écran du Driver Hub.

• Voir en direct ce que fait le programme au lieu de deviner : si les valeurs changent mais que le moteur ne tourne pas, le problème est matériel (câble, port, configuration), pas dans le code.

• Afficher systématiquement les valeurs des capteurs, l'état courant de la machine à états et les consignes : c'est ce qui permet de régler un PID ou de comprendre un autonome qui dérape.

Le lien avec le monde professionnel

• Dans une usine, les techniciens suivent en direct les mesures des machines (vitesses, températures, alarmes) sur des écrans de supervision. La télémétrie, c'est le même principe : on ne dépanne bien que ce que l'on mesure.

3.5. Sauvegarder la configuration et les programmes

- **Programmes** : les télécharger régulièrement sur un ordinateur (Download Op Mode en Blocks, fichier .java en OnBot Java), surtout avant une modification importante et avant chaque compétition. Procédure : gérer les programmes Blocks (FIRST).
- **Configuration** : c'est un fichier stocké dans le dossier « FIRST » du Control Hub. Le copier sur un ordinateur (câble USB-C) permet de le sauvegarder, de le versionner avec le code ou de l'installer sur un autre robot. Procédure Windows et Mac : accéder aux fichiers du Control Hub (REV).

4. Les capteurs

Un programme qui ne mesure rien ne peut rien corriger. Les capteurs donnent au robot l'information dont il a besoin pour savoir où il est, ce qu'il a fait et ce qui l'entoure.

4.1. Les encodeurs

Un **encodeur** convertit une rotation mécanique en signal électrique. Celui intégré à l'arrière des moteurs HD Hex et Core Hex compte les tours, et même les fractions de tour, de l'axe du moteur. Grâce à lui, le programme sait de combien et à quelle vitesse le moteur a tourné.

Type	Ce qu'il donne	Points forts	Limites
Encodeur relatif (incrémental)	Des impulsions pendant la rotation : une variation de position.	Simple et économique. C'est le cas des encodeurs des moteurs du kit.	Aucune position connue au démarrage : il faut partir d'une position de référence.
Encodeur absolu	Directement la position angulaire (par exemple 30°).	Position disponible dès la mise sous tension.	Plus complexe à fabriquer, plus cher.

- **Branchement** : le câble encodeur (4 fils, connecteur JST PH) va sur le port encodeur qui porte le même numéro que le port moteur. Le connecteur a un détrompeur : ne jamais forcer.
- **Distance parcourue par une roue** : nombre de tours de roue × circonférence de la roue, avec nombre de tours de roue = ticks mesurés ÷ (ticks par tour du moteur × rapport de réduction). Les ticks par tour sont donnés par le fabricant du moteur.
- **Modes utiles** : remettre l'encodeur à zéro (STOP_AND_RESET_ENCODER), réguler la vitesse (RUN_USING_ENCODER) ou aller seul à une position cible (RUN_TO_POSITION).
- **Encodeur externe** : un encodeur monté directement sur un axe (par exemple le Through Bore Encoder REV) mesure la rotation d'un bras ou d'une roue d'odométrie.

Pour aller plus loin : les encodeurs et convertir les ticks en distance (REV), les encodeurs (Game Manual 0).

4.2. L'IMU

L'**IMU** (centrale inertielle), intégrée au Control Hub, associe un accéléromètre, un gyroscope et, selon les modèles, un magnétomètre. Par fusion de ces mesures, elle donne directement l'orientation du robot :

- **Yaw** (lacet) : rotation autour de l'axe vertical, c'est-à-dire le cap du robot. C'est l'angle le plus utilisé : maintenir un cap, tourner de 90°.
- **Pitch** (tangage) : inclinaison avant-arrière.
- **Roll** (roulis) : inclinaison gauche-droite.

Les trois pièges de l'IMU

• **L'orientation du Hub** : les axes dépendent de la façon dont le Control Hub est monté dans le robot. Il faut déclarer cette orientation dans le programme : voir orienter l'IMU (REV).

• **L'initialisation** : le robot doit être immobile pendant l'initialisation de l'IMU.

• **La dérive** : le gyroscope dérive lentement avec le temps. La fusion des capteurs limite ce défaut sans l'annuler ; on peut recaler le cap à un moment connu du match.

4.3. Les autres capteurs

Capteur	Port du Hub	Usage typique
Capteur de contact	Numérique	Détecter une butée, savoir si un mécanisme est en position de repos.
Interrupteur magnétique	Numérique	Fin de course sans contact : remettre à zéro l'encodeur d'un ascenseur ou d'un bras.
Capteur de couleur	I2C	Reconnaître une pièce de jeu ou une ligne au sol.
Capteur de distance	I2C	Mesurer la distance à un mur ou à une pièce de jeu.
Potentiomètre	Analogique	Mesurer un angle absolu, par exemple celui d'un bras.

Ressources : introduction aux capteurs, utiliser des capteurs d'autres marques (REV) et lire et écrire sur le matériel (Game Manual 0).

Lire les capteurs efficacement

• Lire chaque capteur une seule fois par tour de boucle et garder la valeur dans une variable.

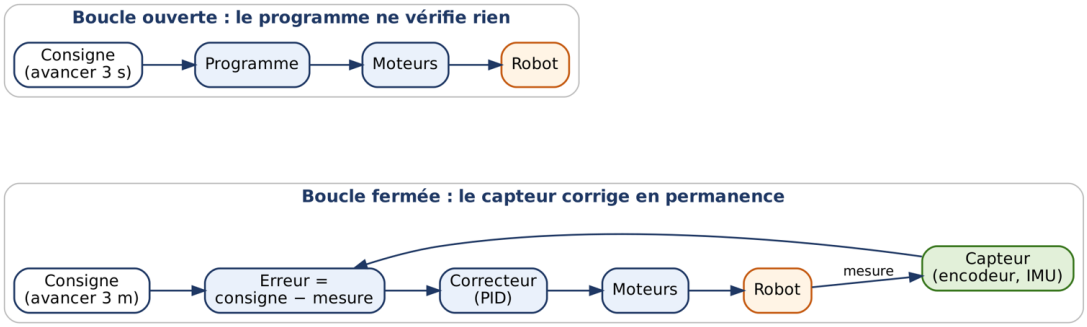
• En Java, activer la lecture groupée (bulk reads) : le programme récupère toutes les valeurs du Hub en une seule fois et la boucle va plus vite.

5. Le contrôle en boucle fermée (PID)

5.1. Pourquoi mesurer pour corriger

Pour faire avancer le robot de 3 m, la première idée est d'allumer les moteurs à mi-puissance pendant un temps donné. Sur plusieurs essais, le robot ne s'arrête ni à 3 m, ni deux fois au même endroit : frottements, jeu dans les transmissions, niveau de la batterie, glissement des roues perturbent le mouvement.

La solution est la **rétroaction** (feedback) : un capteur mesure en permanence ce qui se passe réellement, et le programme corrige la commande en fonction de l'écart avec la consigne.



Boucle ouverte et boucle fermée

5.2. Du tout ou rien au PID

- **Tout ou rien** : pleine puissance tant que la consigne n'est pas atteinte, puis arrêt. Emporté par son élan, le robot dépasse la cible.
- **Proportionnel (P)** : puissance = Kp × erreur. Le robot ralentit en approchant de la cible. Il reste souvent une petite erreur statique ; si elle est négligeable pour la précision visée, inutile d'aller plus loin.
- **PID** : on ajoute deux termes pour corriger ce que le P ne sait pas faire.

Terme	Rôle	Effet d'un réglage trop fort
Kp (proportionnel)	Corrige l'erreur instantanée : augmente la rapidité.	Oscillations autour de la cible.
Ki (intégral)	Accumule l'erreur dans le temps : élimine l'erreur statique.	Système lent à se stabiliser, voire instable.
Kd (dérivé)	Réagit à la vitesse de variation de l'erreur : amortit et stabilise.	Bruit amplifié, réponse lente.

Commande = Kp × erreur + Ki × somme des erreurs + Kd × variation de l'erreur.

5.3. Régler les constantes

Chaque robot a ses propres constantes : celles d'une autre équipe ne fonctionneront pas telles quelles. La méthode empirique la plus utilisée :

1. **Régler Kp** : mettre Ki et Kd à 0, augmenter Kp progressivement jusqu'à ce que le système oscille, puis revenir légèrement en arrière.
2. **Régler Ki** : l'augmenter lentement jusqu'à faire disparaître l'erreur statique, sans créer d'oscillations lentes. Le garder le plus petit possible.
3. **Régler Kd** : l'augmenter progressivement pour réduire les dépassements et amortir les oscillations.

Conseils de réglage

• Toujours régler dans l'ordre Kp, Ki, Kd, en modifiant une seule constante à la fois.

• Commencer avec des valeurs faibles et tester avec un changement brusque de consigne.

• Afficher la consigne, la mesure et l'erreur en télémétrie pendant le réglage.

• Méthode plus rapide mais souvent agressive : Ziegler-Nichols. On augmente Kp seul jusqu'à des oscillations permanentes (gain Ku, période Tu), puis Kp = 0,6 Ku, Ki = 2 Kp ÷ Tu et Kd = Kp × Tu ÷ 8, à affiner ensuite.

Un PID est déjà intégré au Hub

• Les modes RUN_USING_ENCODER et RUN_TO_POSITION utilisent une régulation intégrée au Hub : pour une vitesse de moteur ou une position simple, pas besoin d'écrire son PID.

• Écrire son propre correcteur sert pour ce que ces modes ne couvrent pas : maintenir un cap avec l'IMU, un bras soumis à la gravité, un lanceur à vitesse très stable, un déplacement sur le terrain.

• Pour aller plus loin : boucles de contrôle (Game Manual 0).

Le lien avec le monde professionnel

• La régulation est partout : régulateur de vitesse d'une voiture, stabilisation d'un drone, four qui maintient sa température. Les mêmes trois termes P, I et D y sont réglés par des ingénieurs en automatique.

6. Le mode autonome

En période autonome, pas de manette : tout doit s'enchaîner automatiquement. Un bon autonome se construit par étapes, du plus simple au plus fiable.

6.1. Les timers

Un **timer** mesure le temps écoulé et déclenche une action après un délai : avancer pendant 3 secondes, puis s'arrêter. C'est le plus simple à écrire (objet ElapsedTime), mais le résultat dépend de la charge de la batterie et des frottements : à réserver aux actions dont la précision compte peu, ou comme sécurité (arrêter une action qui dure trop longtemps).

Ressource : ElapsedTime (REV).

6.2. Se déplacer avec les encodeurs et l'IMU

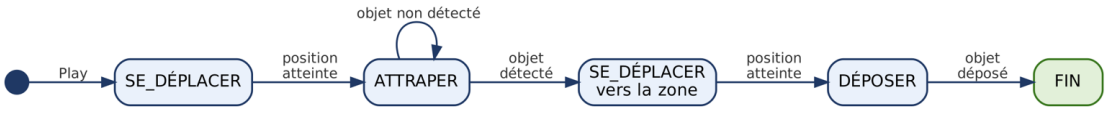
- **Avancer d'une distance** : convertir la distance en ticks, puis utiliser RUN_TO_POSITION.
- **Tourner d'un angle** : soit avec les encodeurs des roues, soit, plus précis, avec le cap donné par l'IMU et un correcteur proportionnel.
- **Garder le cap en ligne droite** : corriger la puissance des côtés gauche et droit selon l'écart de cap mesuré par l'IMU.

Ressources : encodeurs de la base roulante et tourner avec RUN_TO_POSITION (REV).

6.3. Organiser le programme : la machine à états

Une **machine à états** organise le comportement du robot en étapes clairement nommées :

- **État** : la situation dans laquelle se trouve le programme (se déplacer, attraper, déposer).
- **Transition** : le passage d'un état à un autre.
- **Événement ou condition** : ce qui déclenche la transition (position atteinte, capteur qui détecte un objet, timer écoulé, bouton pressé).



Exemple de machine à états pour un mode autonome

À chaque tour de boucle, le programme regarde l'état courant, exécute l'action correspondante et vérifie la condition de sortie. Il ne reste jamais bloqué dans une attente : il peut donc surveiller un capteur, mettre à jour la télémétrie ou faire bouger deux mécanismes en même temps. La même technique sert en TeleOp pour automatiser une séquence déclenchée par un seul bouton.

Ressource : machines à états finis (Game Manual 0).

6.4. Savoir où est le robot : l'odométrie

L'**odométrie** estime la position du robot sur le terrain (x, y) et son orientation, à partir des capteurs de rotation des roues.

- **Base à deux roues motrices** : la distance parcourue est la moyenne des distances des roues gauche et droite ; la rotation est la différence de ces distances divisée par l'écart entre les roues. Simple, mais faussé dès qu'une roue glisse.
- **Trois roues d'odométrie** : de petites roues non motrices, montées sur des encodeurs externes et plaquées au sol. Deux roues parallèles mesurent l'avance et la rotation, une roue perpendiculaire mesure le déplacement latéral. Beaucoup plus précis, et indispensable avec une base mecanum.
- **Bibliothèques de suivi de trajectoire** : Road Runner et Pedro Pathing calculent la position et font suivre au robot des trajectoires courbes. Ces bibliothèques Java prêtes à l'emploi s'utilisent avec Android Studio ; en Blocks, on peut programmer sa propre odométrie et son suivi de trajectoire.

Ressources : odométrie et cinématique (Game Manual 0).

Construire son autonome pas à pas

• Commencer simple : un autonome qui marque quelques points de façon fiable vaut mieux qu'un autonome ambitieux qui échoue une fois sur deux.

• Fiabiliser ensuite avec les encodeurs et l'IMU, puis structurer avec une machine à états.

• Tester chaque version au moins dix fois et noter le taux de réussite.

• Prévoir un programme par position de départ et par couleur d'alliance, avec des noms explicites pour que le pilote choisisse le bon sur le Driver Hub.

7. La vision par ordinateur

7.1. Les AprilTags

Les **AprilTags** sont des codes-barres en deux dimensions placés sur le terrain. À partir de l'image d'une webcam, le programme lit le numéro de chaque tag et calcule sa position et son orientation par rapport à la caméra. Le robot peut ainsi reconnaître une configuration du terrain ou se recaler sur une position connue.

Le SDK fournit **VisionPortal**, qui gère la caméra et la détection. Des programmes d'exemple sont disponibles **en Blocks comme en Java** : la vision n'est pas réservée aux équipes sous Android Studio.

Ressources : introduction aux AprilTags, VisionPortal, comprendre les valeurs de détection, se localiser avec les AprilTags (FIRST) et vision par ordinateur (Game Manual 0).

7.2. Choisir une webcam

Vous trouverez ici un récapitulatif des webcams régulièrement utilisées par les équipes pour la détection des AprilTags. Les webcams Microsoft et Logitech sont disponibles auprès de nombreux revendeurs et sur les marketplaces de votre choix.

D'autres références, non mentionnées dans ce lien, sont également utilisées par certaines équipes. Toutefois, leur prix est plus élevé que celui des webcams classiques :

- Limelight Vision Limelight 3A
- DFRobot HuskyLens
- Charmed Labs Pixy2

Bonnes pratiques vision

• Fixer la caméra rigidement : si elle bouge, toutes les positions calculées sont fausses.

• Utiliser une résolution pour laquelle la webcam dispose d'une calibration (640 × 480 pour la plupart des modèles du récapitulatif).

• Mesurer et déclarer la position exacte de la caméra sur le robot pour obtenir la position du robot, et pas seulement celle de la caméra.

• Tester dans des conditions proches de la compétition : éclairage, distance, angle de vue.

8. Coder à plusieurs : Git et GitHub

Git enregistre l'historique de toutes les modifications du code. **GitHub** héberge ce code en ligne pour le partager entre les programmeurs de l'équipe. Ensemble, ils règlent la principale difficulté d'une équipe qui grandit : travailler à plusieurs sur le même code sans perdre de travail.

8.1. Les notions à connaître

Terme	Définition
Dépôt (repository)	Le dossier du projet et tout son historique.
Fork	Copie d'un dépôt GitHub sur un autre compte ou une organisation, qui garde le lien avec l'original. L'équipe fait un fork de FtcRobotController pour récupérer plus tard les mises à jour du SDK.
Clone	Copie du dépôt GitHub sur l'ordinateur, où l'on code et teste.
Branche	Une ligne de travail séparée : nouvelle fonctionnalité, correction de bug ou expérimentation, sans toucher au code qui marche.
Commit	Un point d'enregistrement dans l'historique, avec un message qui explique la modification.
Push	Envoyer ses commits de l'ordinateur vers GitHub.
Fetch / Pull	Récupérer les modifications des autres depuis GitHub. Pull = fetch + merge.
Merge (fusion)	Réunir le travail de deux branches.

8.2. Mettre en place l'environnement Android Studio

Objectif : récupérer le projet officiel FtcRobotController dans l'organisation GitHub de l'équipe, puis l'ouvrir dans Android Studio sur chaque ordinateur.

Le code appartient à l'équipe, pas à un élève

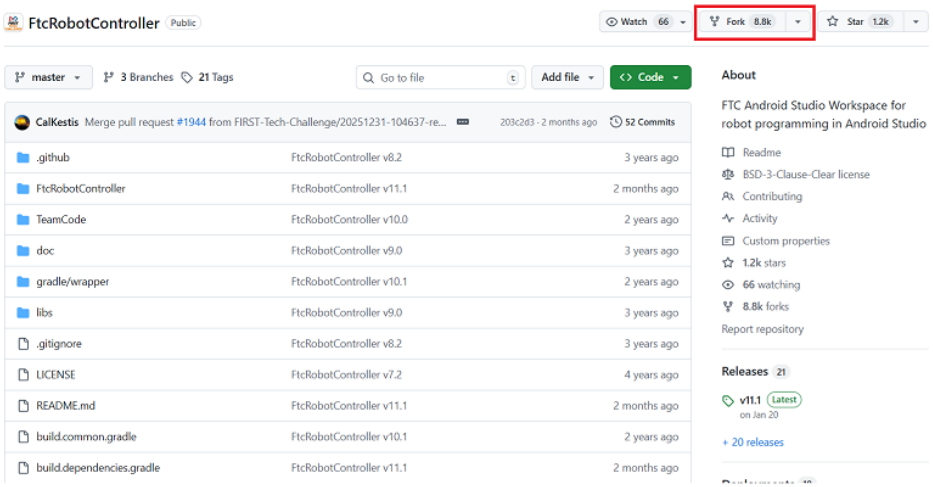
• Le dépôt est créé dans une **organisation GitHub de l'équipe**, gérée par le coach, et non sur le compte personnel d'un élève. Quand un élève quitte l'équipe, le code, l'historique et les accès restent à l'équipe.

• Les élèves utilisent leur propre compte GitHub et sont invités comme membres de l'organisation.

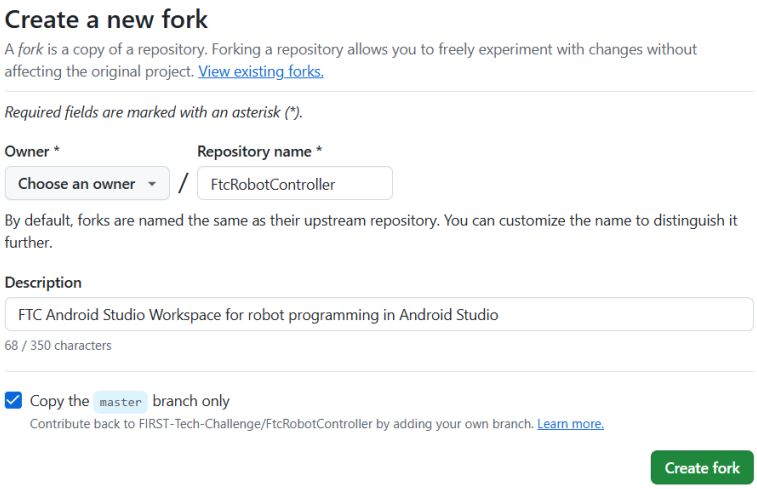
Étape 1 (coach) : créer l'organisation et le dépôt de l'équipe

1. Créer un compte sur GitHub si ce n'est pas déjà fait, puis créer une organisation pour l'équipe (offre gratuite), par exemple « FTC-XXXXX ».
2. Ouvrir le dépôt officiel FtcRobotController.

3. Cliquer sur le bouton **Fork**, en haut à droite.
4. Dans la fenêtre qui s'ouvre, choisir l'organisation de l'équipe dans **Owner**, garder ou modifier le nom du dépôt, puis cliquer sur **Create fork**. L'équipe a maintenant sa propre copie du projet.
5. Inviter chaque programmeur comme membre de l'organisation, avec un droit d'écriture sur le dépôt.



Le bouton Fork sur la page du dépôt FtcRobotController



La fenêtre de création du fork

Étape 2 (chaque programmeur) : copier l'adresse du dépôt

1. Accepter l'invitation reçue, puis ouvrir le dépôt de l'équipe sur GitHub.

2. Cliquer sur le bouton **Code**, choisir **HTTPS** et copier l'adresse, par exemple `https://github.com/FTC-XXXXX/FtcRobotController.git`.

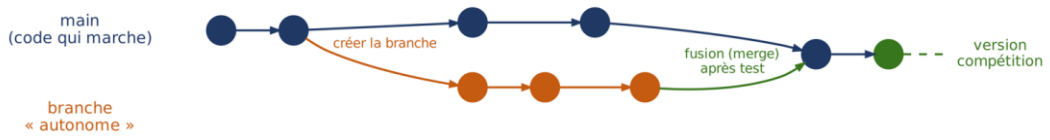
Étape 3 : cloner le projet dans Android Studio

1. Installer Android Studio si ce n'est pas déjà fait : procédure d'installation (FIRST).
2. Vérifier que Git est installé : sinon, Android Studio propose de l'installer automatiquement.
3. Ouvrir le menu ☰ en haut à gauche, puis **Git**, puis **Clone**.
4. Coller l'adresse copiée sur GitHub et choisir un dossier de destination sur l'ordinateur.
5. Valider : le projet se télécharge et s'ouvre automatiquement.

Étape 4 : se connecter à GitHub. Lors du clonage ou du premier envoi (push), Android Studio demande de se connecter au compte GitHub : suivre les instructions (authentification dans le navigateur ou par jeton).

8.3. Une organisation qui fonctionne

1. Créer une branche par fonctionnalité : « autonome-gauche », « lanceur », « vision ».
2. Faire des commits petits et fréquents, avec un message clair (« Ajout du maintien de cap en autonome »).
3. Tester sur le robot avant de fusionner dans la branche principale.
4. La branche principale contient toujours du code qui marche. Marquer la version utilisée en compétition pour pouvoir y revenir.
5. Récupérer chaque saison les mises à jour du SDK depuis le dépôt d'origine (fetch, puis merge).



Une branche par fonctionnalité, fusionnée dans la branche principale après test

Le piège de la fusion réussie

• Une fusion peut se dérouler sans conflit et donner pourtant un résultat faux. Exemple : deux élèves ajoutent la même méthode, l'un en haut du fichier, l'autre en bas. Git ne voit aucun chevauchement et garde les deux copies.

• Après chaque fusion : compiler, relire les changements et tester sur le robot.

Et en Blocks ou en OnBot Java ?

• Désigner un responsable par programme : une seule personne modifie un programme donné.

• Télécharger régulièrement les programmes et les ranger dans un dossier partagé ou un dépôt GitHub, avec la date : c'est un historique manuel, mais il sauve une compétition.

Le lien avec le monde professionnel

• En entreprise, aucun code n'est modifié directement sur la version en production : chaque changement passe par une branche, une relecture par un collègue (pull request) et des tests automatiques avant d'être fusionné.

9. Dépannage

Problème	Pistes à vérifier
Le Driver Hub ne se connecte pas au Control Hub	Control Hub allumé (voyant vert), bon réseau Wi-Fi et bon mot de passe.
Un moteur ne tourne pas	Port de branchement, nom dans la configuration active, programme bien enregistré.
Un moteur tourne à l'envers	Inverser le sens du moteur dans le programme (Direction : REVERSE).
Un servomoteur ne bouge pas ou force	Port, type déclaré (Servo ou Continuous Rotation Servo), position commandée, sens du câble.
Le programme affiche une erreur de configuration	Un nom du programme ne correspond à aucun nom de la configuration active : vérifier l'orthographe et les majuscules.
Les blocs d'un moteur n'apparaissent pas dans l'éditeur Blocks	Aucune configuration active : créer et activer la configuration avant de coder.
Un encodeur renvoie 0 ou des valeurs incohérentes	Câble encodeur sur le port du même numéro que le moteur, remise à zéro de l'encodeur à l'initialisation.
L'autonome ne s'arrête jamais au même endroit	Charge de la batterie ; remplacer les timers par les encodeurs et l'IMU (chapitre 6).
L'angle de l'IMU est faux	Orientation du Hub déclarée dans le programme, robot immobile pendant l'initialisation.
Aucun AprilTag détecté	Webcam branchée et déclarée dans la configuration, résolution calibrée, éclairage, distance au tag.
Comportements incohérents (moteurs lents, déconnexions)	Batterie chargée et solidement branchée.

Pour aller plus loin : les journaux du Control Hub, les codes de clignotement du voyant et le dépannage général (REV). En cas de doute, contacter le conseiller technique Robotique FIRST France.

Ressources supplémentaires

- Les playlists de Brogan M. Pratt : des vidéos de tutoriels FTC (en anglais).
- Pour apprendre la syntaxe Java, de nombreuses ressources existent sur internet, par exemple Apprenez à programmer en Java (OpenClassrooms, gratuit, en français).
- Ressources de programmation FIRST : tutoriels Blocks, OnBot Java et Android Studio, AprilTags, sujets avancés (en anglais).
- Documentation REV du système de contrôle : Control Hub, capteurs et guides Hello Robot en Blocks et en Java (en anglais).
- Game Manual 0, partie logiciel : la référence de la communauté FTC, en français.
- FtcRobotController : le projet officiel à utiliser avec Android Studio.
- FTC Sim : des cours guidés pour commencer à apprendre la programmation FTC.

☑ Pour commencer dès maintenant

- Choisir l'outil de programmation de la saison selon le nombre de programmeurs et les besoins (chapitre 1).
- Mettre à jour le Driver Hub et le Control Hub, et vérifier qu'ils ont la même version.
- Sauvegarder la configuration et tous les programmes actuels sur un ordinateur.
- Ajouter la télémétrie des capteurs dans le programme TeleOp.
- Remplacer les timers de l'autonome par les encodeurs et l'IMU, puis le structurer en machine à états.
- Si plusieurs élèves codent : créer l'organisation GitHub de l'équipe et y faire le fork de FtcRobotController.

Le conseiller technique Robotique FIRST France reste disponible pour accompagner l'équipe sur ces étapes.